Name: _____

Quiz 4

---

1. Shown below is a chained hash table of size 7 containing some keys. Each key is a lowercase string and each cell can hold multiple keys. The index of each cell in the table is also shown.

a idea	habib i biryani	logic mj	ok speed	have liberal fries	rahim	pehlwan egg
0	1	2	3	4	5	6

   a. [2 points] Study the distribution of keys in the table and, for each of the keys “yohsin" and “university", indicate the index it will hash to according to the established pattern. Assume that there is no need for resize.

      “yohsin” : _____

      “university” : _____

   b. [1 points] Provide pseudocode for a hash function that efficiently implements the hashing pattern observed in the table above. Assume that the input consists of lowercase strings in the English alphabet only.

   c. [2 points] Consider an empty hash table of size 7 that uses open addressing (linear probing) for conflict resolution. Using the hash function that you defined in b, indicate the index in the table for each of the words in the following sentence. The words are added in the order of appearance in the sentence.

      “python is a language of consenting adults”

Name: _____

d. [2 points] Consider the following hash function:

```
possible_hash (s)
    if len(s) < 1 then return len(s)
    else   s1 = s.substr(0, len(s)-1) //drop last letter, e.g. if s is "hash", then s1 is " has"
    return possible_hash (s1)
```

Which collision resolution method (Separate Chaining or Linear Probing), could implement a find(key) operation in O(1) expected time on a hash table that utilizes possible_hash? Justify your choice.

2. [3 points] Consider a hash table that uses Separate Chaining and also keeps track of the number of keys that it contains. It stores each key at most once; adding a key a second time has no effect. It takes the steps necessary to ensure that the number of keys is always less than or equal to twice the number of buckets (i.e., that the load factor is ≤ 2). Assume that its hash function and comparison of keys take constant time. All bounds should be a function of *N*, the number of elements in the table.

   a. Give Θ() bounds on the worst-case time of adding an element to the table.

   b. Assume that the hashing function is so good that it always evenly distributes keys among buckets. What are the Θ() bounds on the worst-case time of adding an element?

   c. Making no assumption about the goodness of the hashing function, suppose that instead of using linked lists for the buckets, we use some kind of Skiplist that somehow keeps itself “balanced”. What bound can you place on the Θ() worst-case time for testing to see if an item is in the table?